COMICCRAFT PROJECT DOCUMENTATION

Phase 3: Backend & API Development

System Architecture, API Endpoints, and AI Service Integration

1. Phase Overview

The third phase of the ComicCraft project focuses on developing the backend system and APIs. The backend acts as the core connection bridge between the frontend application, AI services, and database/storage infrastructure.

2. Objectives

The primary objectives of this phase are to:

- Set up the core ComicCraft backend server.
- Design and construct robust REST APIs.
- Establish seamless communication between frontend and backend systems.
- Handle and sanitize incoming user requests.
- Integrate external AI story and image generation services securely.
- Manage end-to-end comic generation requests and lifecycle.
- Implement centralized error handling and structured reporting.

3. Backend Technology Stack

The backend architecture is built using modern, scalable JavaScript tools:

Node.js **Express.js** **JavaScript (ES6+)** **RESTful Architecture** **Dotenv (.env)**

4. Backend Directory Structure

The project follows a modular and maintainable folder structure:

```
backend/
│
├── server.js               # Application entry point
├── routes/
│   ├── comicRoutes.js      # Routes for comic generation
│   └── healthRoutes.js     # Health check status routes
│
├── controllers/
│   └── comicController.js  # Request processing logic
│
├── services/
│   ├── aiService.js        # Text/Story generation AI logic
│   └── imageService.js     # Image generation AI service
│
├── models/
│   └── comicModel.js       # Data models and schema definitions
│
├── config/
│   └── config.js           # Configuration and environment variables
│
├── middleware/
│   └── errorHandler.js     # Global error handling middleware
│
└── .env                    # Environment secrets (API Keys)
```

5. API Specifications

HEALTH CHECK API

Monitors server availability and performance health.

```
GET /api/health
```

COMIC GENERATION API

Receives prompt parameters from the frontend to trigger comic creation.

```
POST /api/comic/generate
```

Example Request Payload:

```
{
  "prompt": "A young hero discovers a mysterious city",
  "style": "cartoon",
```

```
  "panels": 4
}
```

6. AI Service Integration & Workflow

The backend orchestrates communication between the UI and external AI services in a sequential pipeline:

Frontend → Backend API → AI Service → Generated Story → Image Generation Service → Comic Panels → Backend → Frontend

Security Note: API keys are securely stored in environment variables (`.env`) on the server side and are never exposed to the client-side code.

7. Request Execution Flow

When a user requests a comic, the backend executes the following steps:

1. The frontend submits the prompt payload to the backend endpoint.
2. The backend validates and sanitizes input data.
3. The prompt is routed to the AI text generation service.
4. The AI generates structured story scripts and panel prompts.
5. Panel prompts are sent to image generation APIs.
6. The compiled panel assets and metadata are returned to the frontend.

8. Error Handling & Security

Centralized middleware intercepts and handles system issues such as invalid user input, missing fields, AI service downtime, image generation failures, and internal server faults.

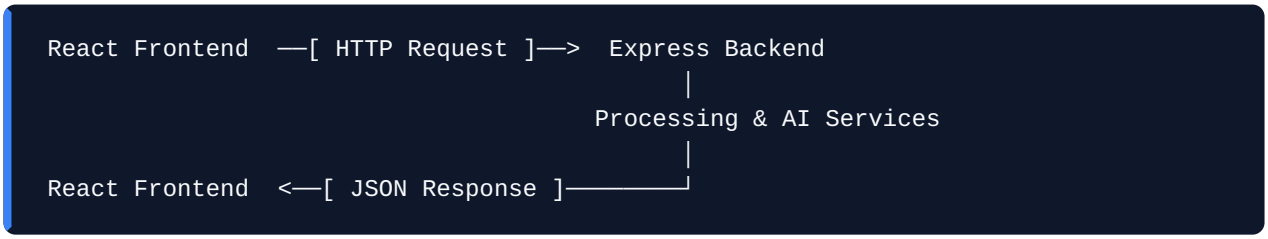
Standardized Failure Response:

```
{
  "success": false,
```

```
  "message": "Comic generation failed. Please try again."
}
```

Security Best Practices Implemented:

- Store secret keys strictly in server-side `.env` files.
- Validate and sanitize all incoming payload data.
- Configure strict Cross-Origin Resource Sharing (CORS) rules.
- Prevent leaking stack traces or sensitive errors to the frontend.

9. Client-Server Communication Flow

Communication between the React frontend and Express backend uses RESTful HTTP protocols over JSON.